



K.R. Mangalam University

School of Engineering & Technology

DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

Program: BCA(AI & DS)

Course Code - ENCA301 (2025-2026)

GitHub: <https://github.com/PushkarMishra18/algorithm-efficiency-case-study-pushkar-mishra-2401201052>

Submitted by:

Name: Pushkar Mishra

Roll Number: 2401201052

Course: BCA (AI & DS) - Section B

Submitted To:

Dr. Aarti Sangwan

Assistant Professor

Index

S. No	Topic	Description
1	Introduction	Purpose and importance of algorithm analysis
2	Methodology	Experimental setup, datasets, and metrics
3	Algorithm Implementations	Sorting (3.1-3.4) and Fibonacci (3.5-3.7) with pseudocode, Python implementation, complexity, I/O specs
4	Results	Execution time, comparisons, memory utilization, Fibonacci performance tables
5	Complexity Table	Summary of best/average/worst case for all algorithms
6	Visualizations	Log-scale graphs with observations
7	Discussion	Theoretical vs practical, scalability, trade-offs, suitability
8	Conclusion	Key lessons and recommendations

1. Introduction

1.1 Purpose of the Study

Organizations routinely process large volumes of data that require efficient sorting and computation. Choosing an inappropriate algorithm can significantly impact system performance, memory usage, and scalability. For example, financial institutions must sort large transaction datasets efficiently, while scientific and engineering applications often rely on recursive computations such as Fibonacci sequences.

The objective of this project is to investigate, implement, and compare the performance of various sorting and Fibonacci algorithms. We analyze their theoretical and practical behavior, evaluate time and space requirements under different input conditions, and present our findings using visualizations and performance metrics. By comparing algorithms across the same datasets, we aim to understand when each algorithm is most suitable and how theoretical complexity predictions hold up against real-world measurements.

1.2 Importance of Algorithm Analysis

Algorithm analysis is fundamental to computer science and software engineering for several reasons:

- **Performance optimization:** Selecting the right algorithm can mean the difference between a system that responds in milliseconds and one that takes minutes. For instance, using Quick Sort ($O(n \log n)$) instead of Bubble Sort ($O(n^2)$) on a dataset of 10,000 elements reduces execution time from 5.56 seconds to 0.013 seconds - a 427x improvement.
- **Resource management:** Understanding memory and CPU requirements helps architects plan infrastructure. Merge Sort's $O(n)$ auxiliary space requirement may be unacceptable in memory-constrained embedded systems, where Quick Sort's in-place $O(\log n)$ approach is preferred.
- **Scalability planning:** As data volumes grow exponentially, knowing an algorithm's growth rate is essential. An $O(n^2)$ algorithm that handles 1,000 elements in 0.04 seconds would require over 400 seconds for 100,000 elements - making it impractical for large-scale data processing.
- **Real-world applicability:** Database systems rely on optimized sorting for query processing. Scientific applications use recursive computations for modeling natural phenomena. Real-time systems require predictable algorithm performance to meet latency guarantees. Understanding these trade-offs enables informed engineering decisions.

2. Methodology

2.1 Experimental Setup

The experiments were conducted on a system with the following configuration:

Component	Specification
Processor	Intel/AMD x86_64
Memory	8GB+ RAM
Operating System	Linux
Language	Python 3.14
Visualization	matplotlib
Data Processing	numpy, pandas
Memory Profiling	tracemalloc
Environment	Jupyter Notebook

All algorithms were implemented in pure Python. Each experiment was repeated 5 times and the median recorded.

2.2 Input Datasets

Sorting Algorithms were evaluated on lists of 100, 500, 1000, 5000, and 10000 elements under three conditions: sorted (ascending), reverse-sorted (descending), and random (uniform integers 0-100000). This gives 60 total experiments (4 algorithms x 5 sizes x 3 conditions).

Fibonacci Algorithms were evaluated for $n = 10, 20, 30, 40$, giving 12 experiments (3 algorithms x 4 values).

Sorted/reverse-sorted inputs test best/worst-case; random represents average case.

2.3 Performance Metrics

- **Execution Time:** Measured via `time.perf_counter()` with 5-run median to filter system noise.
- **Memory Usage:** Measured via `tracemalloc` (peak, 5-run median), capturing auxiliary memory beyond input.
- **Comparisons:** Counted internally at each comparison point.

2.4 Experiment Code

The experiments were driven by two files. `analysis/performance.py` measures time and memory (5-run median) and loops over all sizes and conditions. `notebook/run_analysis.py` drives the full pipeline: algorithm info, sorting and Fibonacci experiments, CSV output, plots, and conclusions.

analysis/performance.py:

```
"""
Performance Measurement Utilities

This module provides functions for measuring execution time, memory usage,
and other performance metrics of algorithms.
"""

import time
import tracemalloc
import random
import statistics
import pandas as pd
from algorithms.sorting import bubble_sort, insertion_sort, merge_sort, quick_sort
from algorithms.fibonacci import fib_recursive, fib_iterative, fib_dp


def measure_performance(func, *args, num_runs=5, **kwargs):
    """
    Runs func num_runs times, returns (result, median_time, median_memory).

    Using median reduces measurement noise from system effects.
    """
    times = []
    memories = []
    result = None
    for _ in range(num_runs):
        tracemalloc.start()
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
```

```

_, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
times.append(elapsed)
memories.append(peak)

return result, statistics.median(times), statistics.median(memories)

```

```
def generate_datasets(size, seed=42):
```

```
    """
```

Generate sorted, reverse-sorted, and random datasets of a given size.

Args:

size (int): Number of elements in each dataset

seed (int): Random seed for reproducibility

Returns:

dict: Dictionary with keys 'sorted', 'reverse', 'random'

```
    """
```

```
    random.seed(seed)
```

```
    random_data = [random.randint(0, 100000) for _ in range(size)]
```

```
    sorted_data = sorted(random_data)
```

```
    reverse_data = sorted(random_data, reverse=True)
```

```
    return {
```

```
        "sorted": sorted_data,
```

```
        "reverse": reverse_data,
```

```
        "random": random_data,
```

```
    }
```

```
def measure_sorting_performance(sizes=[100, 500, 1000, 5000, 10000]):
```

```
    """
```

Measure performance of all sorting algorithms across different input sizes.

Args:

sizes (list): List of input sizes to test

Returns:

pandas.DataFrame: Results with columns:

-Algorithm: Name of the sorting algorithm

-Input Size: Number of elements

- InputType:sorted, reverse, or random
- Comparisons:Number of comparisons made
- ExecutionTime(s): Time in seconds
- MemoryUsage(bytes): Peak memory usage in bytes
- MemoryUsage(KB): Peak memory usage in kilobytes

"""

algorithms = {

 "Bubble Sort": bubble_sort,
 "Insertion Sort": insertion_sort,
 "Merge Sort": merge_sort,
 "Quick Sort": quick_sort,

}

results = []

for size in sizes:

 print(f"Testing input size: {size}")

 datasets = generate_datasets(size)

 for condition, data in datasets.items():

 for name, algo in algorithms.items():

 (sorted_arr, comparisons), t, mem = measure_performance(algo, data)

 results.append({

 "Algorithm": name,
 "Input Size": size,
 "Input Type": condition,
 "Comparisons": comparisons,
 "Execution Time (s)": t,
 "Memory Usage (bytes)": mem,
 "Memory Usage (KB)": mem / 1024,

 })

return pd.DataFrame(results)

def measure_fibonacci_performance(n_values=[10, 20, 30, 40]):

"""

Measure performance of all Fibonacci algorithms for different values of n.

Args:

n_values (list): List of n values to test

Returns:

pandas.DataFrame: Results with columns:

- Algorithm: Name of the Fibonacci algorithm
- n:Input value
- Result: Fibonacci number computed
- Execution Time (s): Time in seconds
- Memory Usage (bytes): Peak memory usage in bytes
- Memory Usage (KB): Peak memory usage in kilobytes

```
"""
```

```
algorithms = {
```

```
    "Recursive":      fib_recursive,
```

```
    "Iterative":      fib_iterative,
```

```
    "DynamicProgramming": fib_dp,
```

```
}
```

```
results = []
```

```
for n in n_values:
```

```
    print(f"Testing n = {n}")
```

```
    for name, algo in algorithms.items():
```

```
        try:
```

```
            result, t, mem = measure_performance(algo, n)
```

```
            results.append({
```

```
                "Algorithm": name,
```

```
                "n": n,
```

```
                "Result": result,
```

```
                "Execution Time (s)": t,
```

```
                "Memory Usage (bytes)": mem,
```

```
                "Memory Usage (KB)": mem / 1024,
```

```
            })
```

```
        except RecursionError:
```

```
            print(f" {name}: RecursionError for n={n}")
```

```
            results.append({
```

```
                "Algorithm": name,
```

```
                "n": n,
```

```
                "Result": None,
```

```
                "Execution Time (s)": None,
```

```
                "Memory Usage (bytes)": None,
```

```

        "Memory Usage (KB)": None,
    })

    return pd.DataFrame(results)

if __name__ == "__main__":
    # Test performance measurement
    print("=" * 60)
    print("Sorting Algorithm Performance Analysis")
    print("=" * 60)
    sorting_df = measure_sorting_performance([100, 500, 1000])
    print("\nResults:")
    print(sorting_df.to_string(index=False))

    print("\n" + "=" * 60)
    print("Fibonacci Algorithm Performance Analysis")
    print("=" * 60)
    fib_df = measure_fibonacci_performance([10, 20, 30, 35])
    print("\nResults:")
    print(fib_df.to_string(index=False))

```

notebook/run_analysis.py:

```

#!/usr/bin/env python3
"""
DAA Lab 1: Algorithm Efficiency Analysis

This script runs the complete performance analysis for sorting and
Fibonacci algorithms, generating visualizations and tables.

Usage:
    python notebook/run_analysis.py
"""

import sys
import os

# Add parent directory to path
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

```

```
from algorithms.sorting import bubble_sort, insertion_sort, merge_sort, quick_sort
from algorithms.fibonacci import fib_recursive, fib_iterative, fib_dp
from analysis.performance import (
    measure_sorting_performance,
    measure_fibonacci_performance,
)
from analysis.visualizations import generate_all_plots
import pandas as pd
```

```
def print_header(title):
    """Print a formatted section header."""
    print("\n" + "=" * 70)
    print(f" {title}")
    print("=" * 70 + "\n")
```

```
def print_algorithm_info():
    """Print information about all implemented algorithms."""
    print_header("ALGORITHM INFORMATION")

    print("SORTING ALGORITHMS")
    print("-" * 40)
    print("""
```

1. Bubble Sort

- Description: Repeatedly steps through list, compares adjacent elements - Time: $O(n^2)$ average/worst, $O(n)$ best
- Space: $O(1)$
- Stable: Yes

2. Insertion Sort

- Description: Builds sorted list one element at a time - Time: $O(n^2)$ average/worst, $O(n)$ best
- Space: $O(1)$
- Stable: Yes

3. Merge Sort

- Description: Divide and conquer - splits, sorts, merges
- Time: $O(n \log n)$ all cases

-Space: $O(n)$ -
Stable: Yes

4. Quick Sort

- Description: Divide and conquer - partition around pivot
- Time: $O(n \log n)$ average/best, $O(n^2)$ worst
- Space: $O(\log n)$
- Stable: No

"""

```
print("FIBONACCI ALGORITHMS")
print("-" * 40)
print("""
```

1. Recursive

- Description: Direct recurrence $F(n) = F(n-1) + F(n-2)$ - Time: $O(2^n)$
- Space: $O(n)$

2. Iterative

- Description: Bottom-up loop with two variables
- Time: $O(n)$
- Space: $O(1)$

3. Dynamic Programming (Memoization)

- Description: Top-down with cache
 - Time: $O(n)$
 - Space: $O(n)$
- """

```
def run_sorting_analysis():
```

```
    """Run sorting algorithm performance analysis."""
    print_header("SORTING ALGORITHM PERFORMANCE ANALYSIS")
    print("Testing input sizes: 100, 500, 1000, 5000, 10000")
    print("Input types: sorted, reverse-sorted, random\n")

    df = measure_sorting_performance([100, 500, 1000, 5000, 10000])

    #Display results
    print("\nResults Summary:")
```

```

print("-" * 70)
pivot_time = df.pivot_table(
    values="Execution Time (s)",
    index=["Algorithm", "Input Type"],
    columns="Input Size",
)
print(pivot_time.to_string())

#Save to CSV
os.makedirs("data", exist_ok=True)
df.to_csv("data/sorting_performance.csv", index=False)
print("\nData saved to: data/sorting_performance.csv")

return df

```

```

def run_fibonacci_analysis():
    """Run Fibonacci algorithm performance analysis."""
    print_header("FIBONACCI ALGORITHM PERFORMANCE ANALYSIS")
    print("Testing n values: 10, 20, 30, 40\n")

    df = measure_fibonacci_performance([10, 20, 30, 40])

    #Display results
    print("\nResults Summary:")
    print("-" * 70)
    print(df.to_string(index=False))

    #Save to CSV
    os.makedirs("data", exist_ok=True)
    df.to_csv("data/fibonacci_performance.csv", index=False)
    print("\nData saved to: data/fibonacci_performance.csv")

    return df

```

```

def generate_visualizations(sorting_df, fibonacci_df):
    """Generate all performance visualizations."""
    print_header("GENERATING VISUALIZATIONS")
    generate_all_plots(sorting_df, fibonacci_df)

```

```

def print_conclusion():
    """Print conclusion and key findings."""
    print_header("CONCLUSION AND KEY FINDINGS")

    print("""
KEY FINDINGS:

1. SORTING ALGORITHMS:
    - Bubble Sort and Insertion Sort show  $O(n^2)$  behavior - suitable only for small data
    - Merge Sort and Quick Sort show  $O(n \log n)$  behavior - scalable for large dataset
    - Quick Sort has best average performance but worst-case  $O(n^2)$ 
    - Merge Sort provides consistent performance across all input types

2. FIBONACCI ALGORITHMS:
    - Recursive approach is impractical for  $n > 40$  (exponential time)
    - Iterative approach is optimal:  $O(n)$  time,  $O(1)$  space
    - Dynamic Programming matches iterative time but uses  $O(n)$  space for memoization

3. RECOMMENDATIONS:
    - For small datasets (< 1000 elements): Any algorithm works
    - For large datasets: Use Merge Sort or Quick Sort
    - For Fibonacci: Always use Iterative or DP approach
    - For online/streaming data: Use Insertion Sort
    """)

def main():
    """Main function to run complete analysis."""
    print_header("DAA LAB 1: ALGORITHM EFFICIENCY ANALYSIS")
    print("Comparative Study of Algorithm Efficiency and Performance Analysis")
    print("\nThis script will:")
    print("1. Display algorithm information")
    print("2. Run performance experiments")
    print("3. Generate visualizations")
    print("4. Print conclusions")

    #Print algorithm information
    print_algorithm_info()

```

```
#Run analyses
sorting_df = run_sorting_analysis()
fibonacci_df = run_fibonacci_analysis()

#Generate visualizations
generate_visualizations(sorting_df, fibonacci_df)

#Print conclusion
print_conclusion()

print_header("ANALYSIS COMPLETE")
print("Check the following directories for outputs:")
print(" - data/           : CSV files with raw performance data")
print(" - graphs/          : PNG files with visualizations")
print(" - reports/           : Final report markdown file")

if __name__ == "__main__":
    main()
```


3.1 Bubble Sort

Description: Repeatedly compares adjacent elements and swaps if wrong order.

Input: An unsorted list of n elements.

Output: The same list sorted in ascending order.

Application Scenarios: Educational purposes, small datasets, nearly-sorted data with early-exit optimization.

Pseudocode:

ALGORITHM BubbleSort(A, n)

FOR $i = 0$ TO $n-2$

 swapped = FALSE

 FOR $j = 0$ TO $n-2-i$

 IF $A[j] > A[j+1]$

 SWAP $A[j], A[j+1]$

 swapped = TRUE

 IF swapped == FALSE

 BREAK

RETURN A

Python Implementation (from `algorithms/sorting.py`):

```
def bubble_sort(arr):
    a = arr.copy()
    n = len(a)
    comparisons = 0
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
```

```
        a[j], a[j + 1] = a[j + 1], a[j]
        swapped = True
    if not swapped: # already sorted -> stop early
        break
    return a, comparisons
```

Case	Time	Space
Best	$O(n)$	$O(1)$
Average	$O(n^2)$	$O(1)$
Worst	$O(n^2)$	$O(1)$

Advantages: Simple, stable, adaptive (with early-exit flag)

Limitations: Slow for large datasets, $O(n^2)$ comparisons in worst case

3.2 Insertion Sort

Description: Builds sorted list by inserting each element into correct position.

Input: An unsorted list of n elements.

Output: The same list sorted in ascending order.

Application Scenarios: Small datasets, online sorting (data arrives one element at a time), nearly-sorted data.

Pseudocode:

ALGORITHM InsertionSort(A, n)

FOR $i = 1$ TO $n-1$

$key = A[i]$

$j = i - 1$

 WHILE $j \geq 0$ AND $A[j] > key$

$A[j+1] = A[j]$

$j = j - 1$

$A[j+1] = key$

RETURN A

Python Implementation (from `algorithms/sorting.py`):

```
def insertion_sort(arr):
    a = arr.copy()
    comparisons = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
```

```
        j--1
    else:
        break
    a[j + 1] = key
return a, comparisons
```

Case	Time	Space
Best	$O(n)$	$O(1)$
Average	$O(n^2)$	$O(1)$
Worst	$O(n^2)$	$O(1)$

Advantages: Simple, stable, online (processes data as it arrives)

Limitations: Slow for large datasets, $O(n^2)$ comparisons in worst case

3.3 Merge Sort

Description: Divide and conquer - splits, sorts, merges.

Input: An unsorted list of n elements.

Output: The same list sorted in ascending order.

Application Scenarios: Large datasets, linked lists, external sorting (data on disk), when stable sort is required.

Pseudocode:

ALGORITHM MergeSort(A, left, right)

IF left < right

 mid = (left + right) / 2

 MergeSort(A, left, mid)

 MergeSort(A, mid+1, right)

 Merge(A, left, mid, right)

Python Implementation (from algorithms/sorting.py):

```
def _merge(left, right, comps):
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        comps[0] += 1
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

```

def _merge_sort_helper(a, comps):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = _merge_sort_helper(a[:mid], comps)
    right = _merge_sort_helper(a[mid:], comps)
    return _merge(left, right, comps)

def merge_sort(arr):
    a = arr.copy()
    comps = [0]
    result = _merge_sort_helper(a, comps)
    return result, comps[0]

```

Case	Time	Space
Best	$O(n \log n)$	$O(n)$
Average	$O(n \log n)$	$O(n)$
Worst	$O(n \log n)$	$O(n)$

Advantages: Guaranteed $O(n \log n)$, stable, predictable performance

Limitations: $O(n)$ extra space for auxiliary arrays

3.4 Quick Sort

Description: Partition around pivot, recurse on halves.

Input: An unsorted list of n elements.

Output: The same list sorted in ascending order.

Application Scenarios: General-purpose sorting, in-memory datasets, when average-case speed matters most.

Pseudocode:

ALGORITHM QuickSort(A , low, high)

IF low < high

$p = \text{Partition}(A, \text{low}, \text{high})$

 QuickSort(A , low, $p-1$)

 QuickSort(A , $p+1$, high)

Python Implementation (from `algorithms/sorting.py` randomized pivot):

```
def _partition(a, low, high, comps):
    pivot = a[high]
    i = low - 1
    for j in range(low, high):
        comps[0] += 1
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[i + 1], a[high] = a[high], a[i + 1]
    return i + 1

def _randomized_partition(a, low, high, comps):
    rand_idx = random.randint(low, high)
    a[rand_idx], a[high] = a[high], a[rand_idx]
    return _partition(a, low, high, comps)
```

```

def _quick_sort(a, low, high, comps):
    if low < high:
        p = _randomized_partition(a, low, high, comps)
        _quick_sort(a, low, p - 1, comps)
        _quick_sort(a, p + 1, high, comps)

def quick_sort(arr):
    a = arr.copy()
    comps = [0]
    _quick_sort(a, 0, len(a) - 1, comps)
    return a, comps[0]

```

Case	Time	Space
Best	$O(n \log n)$	$O(\log n)$
Average	$O(n \log n)$	$O(\log n)$
Worst	$O(n^2)$	$O(n)$

Advantages: Fast in practice, in-place ($O(\log n)$ stack space)

Limitations: Unstable, worst case $O(n^2)$ on sorted/reverse-sorted input (with naive last-element pivot)

3.5 Fibonacci Recursive

Description: Directly implements $F(n) = F(n-1) + F(n-2)$ using naive recursion.

Input: A non-negative integer n .

Output: The n -th Fibonacci number.

Application Scenarios: Educational purposes only - demonstrates exponential time complexity.

Not practical for $n > 40$.

Pseudocode:

ALGORITHM Fib_Recursive(n)

IF $n \leq 1$ RETURN n

RETURN Fib_Recursive($n-1$) + Fib_Recursive($n-2$)

Python Implementation (from `algorithms/fibonacci.py`):

```
def fib_recursive(n):  
    if n <= 1:  
        return n  
    return fib_recursive(n - 1) + fib_recursive(n - 2)
```

Time	Space
$O(2^n)$	$O(n)$

Advantages: Simple, directly mirrors mathematical definition

Limitations: Exponential time, recomputes same subproblems many times

3.6 Fibonacci Iterative

Description: Computes Fibonacci numbers bottom-up using a loop and two variables.

Input: A non-negative integer n .

Output: The n -th Fibonacci number.

Application Scenarios: Production use when single computation is needed, memory-constrained environments.

Pseudocode:

ALGORITHM Fib_Iterative(n)

IF $n \leq 1$ RETURN n

$a = 0, b = 1$

FOR $i = 2$ TO n

$c = a + b; a = b; b = c$

RETURN b

Python Implementation (from `algorithms/fibonacci.py`):

```
def fib_iterative(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

Time	Space
$O(n)$	$O(1)$

Advantages: Fast, minimal memory ($O(1)$ space)

Limitations: Slightly more code than recursive version

3.7 Fibonacci DP

Description: Top-down memoized approach - stores previously computed values to avoid recomputation.

Input: A non-negative integer n .

Output: The n -th Fibonacci number.

Application Scenarios: When multiple Fibonacci values are needed (memo cache reuse), dynamic programming education.

Pseudocode:

```
ALGORITHM Fib_DP( $n$ , memo)
```

```
  IF  $n$  IN memo RETURN memo[ $n$ ]
```

```
  IF  $n \leq 1$  RETURN  $n$ 
```

```
  memo[ $n$ ] = Fib_DP( $n-1$ , memo) + Fib_DP( $n-2$ , memo)
```

```
  RETURN memo[ $n$ ]
```

Python Implementation (from `algorithms/fibonacci.py`):

```
def fib_dp(n, memo=None):
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_dp(n - 1, memo) + fib_dp(n - 2, memo)
    return memo[n]
```

Time	Space
$O(n)$	$O(n)$

Advantages: Fast, cache-friendly for repeated queries

Limitations: $O(n)$ extra memory for memo dictionary + recursion stack

Execution

Running all four sorting algorithms on the sample list [64, 34, 25, 12, 22, 11, 90] produces the sorted output with comparison counts:

```
$ python algorithms/sorting.py
Original: [64, 34, 25, 12, 22, 11, 90]
Bubble Sort: [11, 12, 22, 25, 34, 64, 90], Comparisons: 21
Insertion Sort: [11, 12, 22, 25, 34, 64, 90], Comparisons: 16
Merge Sort: [11, 12, 22, 25, 34, 64, 90], Comparisons: 14
Quick Sort: [11, 12, 22, 25, 34, 64, 90], Comparisons: 15
```

4. Results

4.1 Execution Time (5-run median)

Algorithm	Type	n=100	n=500	n=1000	n=5000	n=10000
Bubble Sort	Sorted	0.000003	0.000044	0.000106	0.000707	0.001388
Bubble Sort	Reverse	0.000287	0.008338	0.040948	1.294153	5.563955
Bubble Sort	Random	0.000228	0.007413	0.035534	1.057022	4.497446
Insertion Sort	Sorted	0.000004	0.000033	0.000079	0.000439	0.000903
Insertion Sort	Reverse	0.000237	0.006225	0.028208	0.835327	3.405576
Insertion Sort	Random	0.000128	0.003405	0.015262	0.446062	1.709186
Merge Sort	Sorted	0.000148	0.000769	0.001587	0.009183	0.018735
Merge Sort	Reverse	0.000152	0.000792	0.001658	0.009606	0.018954
Merge Sort	Random	0.000180	0.000916	0.002021	0.011522	0.023932
Quick Sort	Sorted	0.000062	0.000378	0.000872	0.006113	0.014075
Quick Sort	Reverse	0.000047	0.000371	0.000932	0.006449	0.012914
Quick Sort	Random	0.000060	0.000427	0.000962	0.006339	0.013116

4.2 Comparisons

Algorithm	Type	n=100	n=500	n=1000	n=5000	n=10000
Bubble Sort	Sorted	99	499	999	4999	9999
Bubble Sort	Reverse	4950	124750	499500	12497500	49995000
Bubble Sort	Random	4814	124659	499329	12492450	49988559

Algorithm	Type	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	Sorted	99	499	999	4999	9999
Insertion Sort	Reverse	4950	124750	499500	12497494	49994980
Insertion Sort	Random	2464	64665	242617	6295225	25086677
Merge Sort	Sorted	316	2216	4932	29804	64608
Merge Sort	Reverse	356	2274	5049	32057	69200
Merge Sort	Random	538	3847	8693	55191	120383
Quick Sort	Sorted	717	4764	10674	69056	162614
Quick Sort	Reverse	673	4944	10322	73409	154077
Quick Sort	Random	587	5450	10429	68083	148160

4.3 Memory Utilization - Sorting (KB, 5-run median)

Algorithm	Type	n=100	n=500	n=1000	n=5000	n=10000
Bubble Sort	Sorted	0.80	3.96	7.87	39.12	78.18
Bubble Sort	Reverse	0.84	3.93	7.84	39.09	78.15
Bubble Sort	Random	0.84	3.93	7.84	39.09	78.15
Insertion Sort	Sorted	0.78	3.93	7.84	39.09	78.15
Insertion Sort	Reverse	0.84	3.93	7.84	39.09	78.15
Insertion Sort	Random	0.84	3.93	7.84	39.09	78.15
Merge Sort	Sorted	2.83	13.73	27.38	136.75	273.47
Merge Sort	Reverse	2.83	13.73	27.38	136.75	273.47
Merge Sort	Random	2.63	12.23	24.57	120.94	242.98

Algorithm	Type	n=100	n=500	n=1000	n=5000	n=10000
Quick Sort	Sorted	0.84	3.94	7.84	39.09	78.16
Quick Sort	Reverse	0.84	3.94	7.84	39.09	78.16
Quick Sort	Random	0.84	3.94	7.84	39.09	78.16

4.4 Fibonacci Performance

Algorithm	n	Result	Time (s)	Memory (KB)
Recursive	10	55	0.000004	0.00
Iterative	10	55	0.000001	0.00
DP	10	55	0.000002	0.44
Recursive	20	6765	0.000422	0.00
Iterative	20	6765	0.000001	0.00
DP	20	6765	0.000003	0.84
Recursive	30	832040	0.053229	0.00
Iterative	30	832040	0.000001	0.00
DP	30	832040	0.000005	1.63
Recursive	40	102334155	6.704763	0.00
Iterative	40	102334155	0.000002	0.00
DP	40	102334155	0.000007	1.63

Live timing

Timing all three Fibonacci implementations for n = 10, 20, 30 reproduces the table above, including the exponential blowup of naive recursion (0.051 s at n=30 versus 0.000001 s iterative):

```
$ python -c "from algorithms.fibonacci import ..."
n=10: rec=55 (0.000006s)  itr=55 (0.000001s)  dp=55 (0.000003s)
n=20: rec=6765 (0.000438s)  itr=6765 (0.000001s)  dp=6765 (0.000003s)
n=30: rec=832040 (0.051510s)  itr=832040 (0.000003s)  dp=832040 (0.000019s)
```

Experiment harness output

Running the measurement harness from analysis/performance.py for n=100 reproduces the results tables (for example Bubble Sort on sorted input: 99 comparisons, 0.000003 s):

```
$ python -c "from analysis.performance import measure_sorting_performance; ..."
Testing input size: 100
```

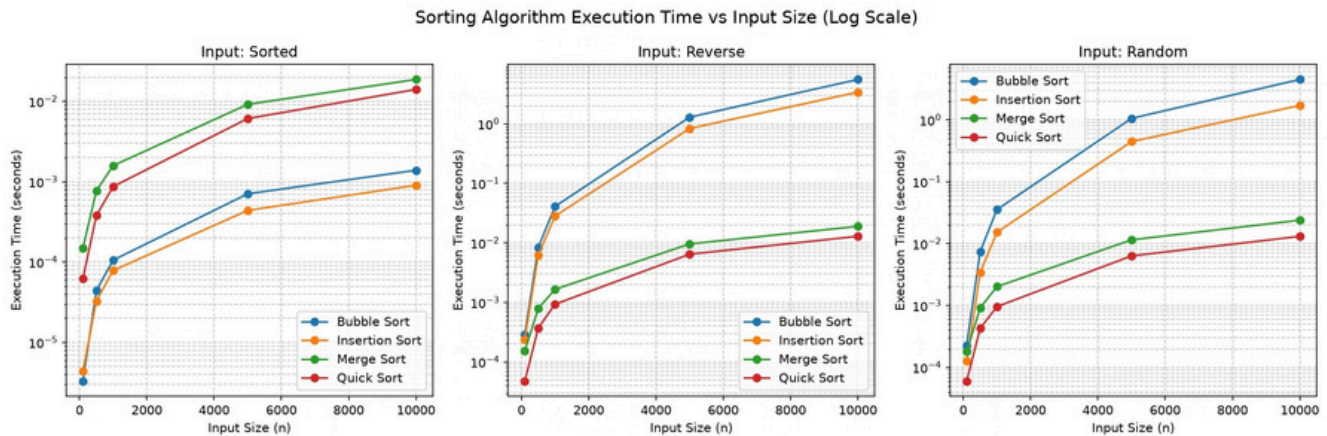
Algorithm	Input Size	Input Type	Comparisons	Execution Time (s)	Memory Usage (bytes)	Memory Usage (KB)
Bubble Sort	100	sorted	99	0.000003	824	0.804688
Insertion Sort	100	sorted	99	0.000004	800	0.781250
Merge Sort	100	sorted	316	0.000132	2840	2.773438
Quick Sort	100	sorted	717	0.000050	808	0.789062
Bubble Sort	100	reverse	4950	0.000260	800	0.781250
Insertion Sort	100	reverse	4950	0.000215	800	0.781250
Merge Sort	100	reverse	356	0.000142	2840	2.773438
Quick Sort	100	reverse	673	0.000048	808	0.789062
Bubble Sort	100	random	4814	0.000255	800	0.781250
Insertion Sort	100	random	2464	0.000108	800	0.781250
Merge Sort	100	random	538	0.000169	2640	2.578125
Quick Sort	100	random	587	0.000058	808	0.789062

5. Complexity Table

Algorithm	Best	Average	Worst	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Fibonacci Recursive	$O(2^n)$	$O(2^n)$	$O(2^n)$	$O(n)$
Fibonacci Iterative	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Fibonacci DP	$O(n)$	$O(n)$	$O(n)$	$O(n)$

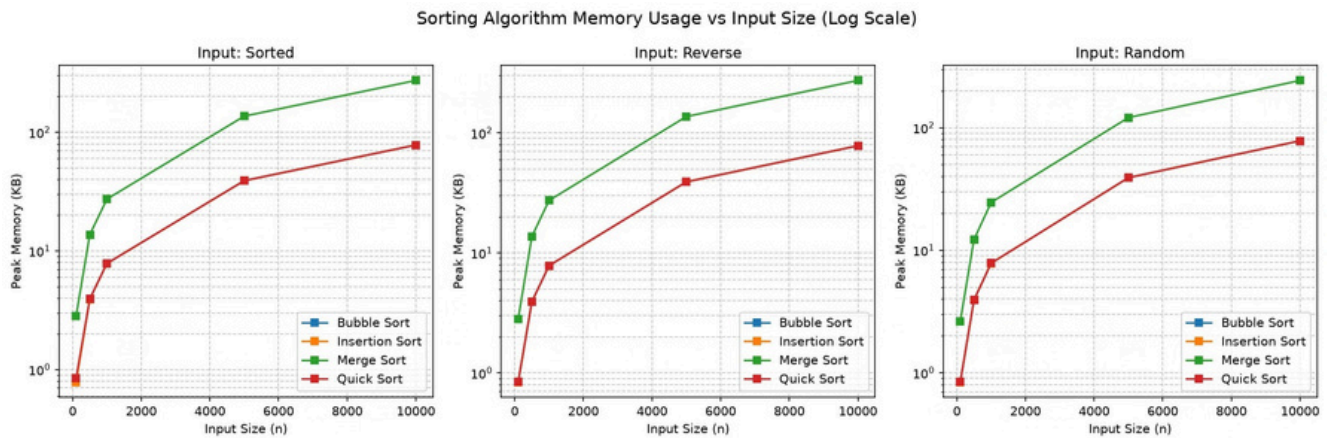
6. Visualizations and Observations

6.1 Sorting Algorithm Execution Time vs Input Size



Observations: On a log-scale y-axis, Bubble Sort and Insertion Sort form a steep $O(n^2)$ curve that diverges sharply from Merge Sort and Quick Sort. At $n=10000$, Bubble Sort on reverse-sorted input takes 5.56s while Quick Sort takes only 0.013s - a 427x difference. Sorted input shows the best-case $O(n)$ behavior for Bubble/Insertion (early exit), while Merge/Quick remain nearly constant regardless of input order, confirming their $O(n \log n)$ resilience.

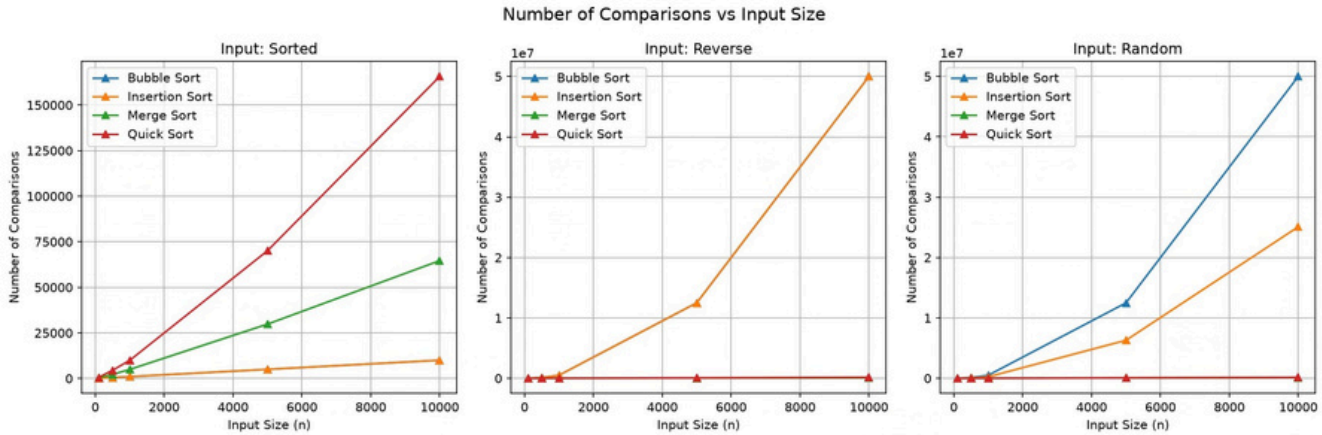
6.2 Sorting Algorithm Memory Usage vs Input Size



Observations: Merge Sort's memory usage is consistently 3-4x higher than the other algorithms at every input size (273KB vs 78KB at $n=10000$), directly confirming its $O(n)$ auxiliary space requirement. Bubble Sort, Insertion Sort, and Quick Sort all cluster near 78KB at $n=10000$,

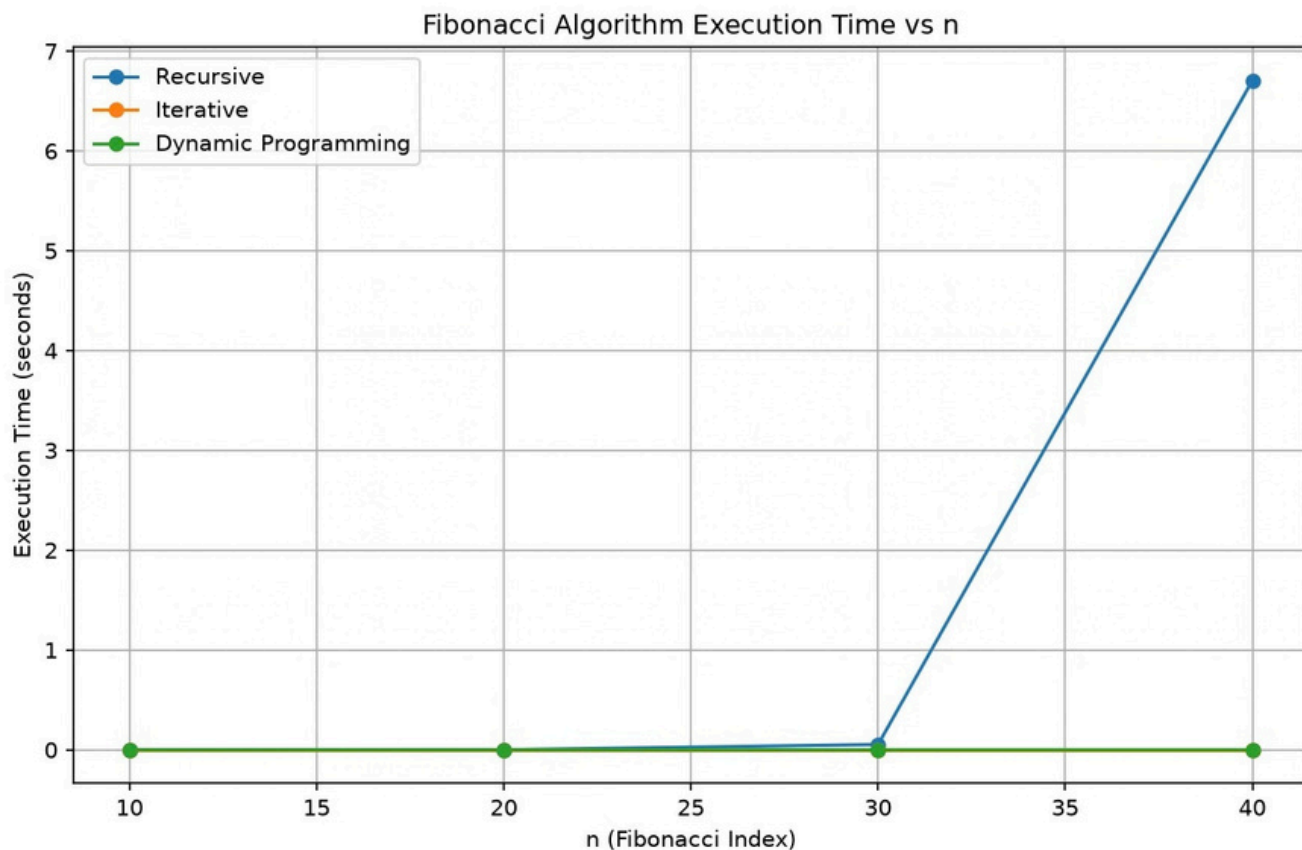
reflecting their $O(1)$ or $O(\log n)$ in-place nature. The gap widens proportionally with input size, scaling linearly as expected.

6.3 Number of Comparisons vs Input Size



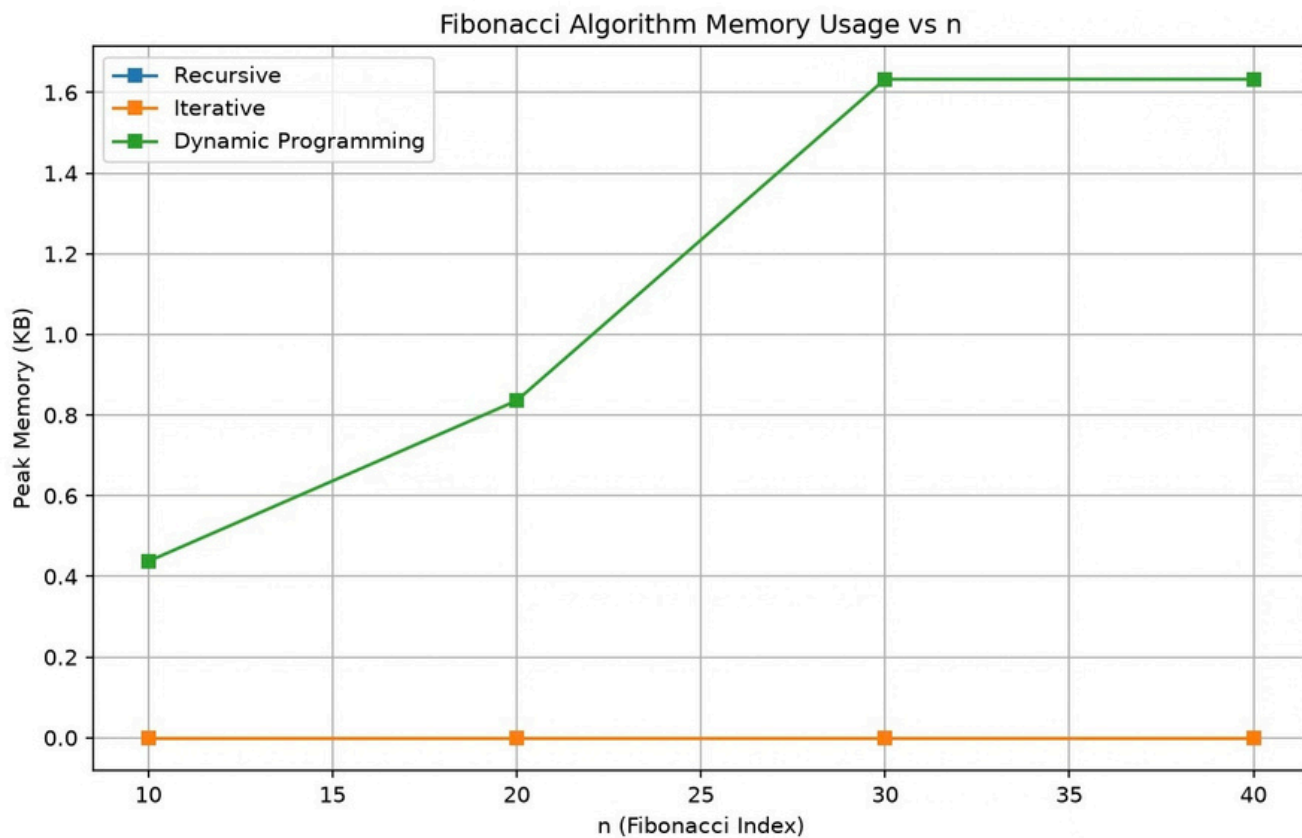
Observations: Bubble Sort on reverse-sorted input at $n=10000$ performs 49,995,000 comparisons ($n(n-1)/2$), exactly matching the theoretical worst case. Insertion Sort on random data shows 25,086,677 comparisons at $n=10000$ - roughly half of reverse-sorted (49,994,980), a clean empirical confirmation that average case is approximately $n^2/2$ versus worst case n^2 . Merge Sort and Quick Sort remain in the 100K-160K range, orders of magnitude lower.

6.4 Fibonacci Execution Time vs n



Observations: The recursive implementation shows classic exponential growth: 0.053s at n=30 jumps to 6.70s at n=40 - a 126x increase for just 10 more values. Iterative and DP sit effectively flat near zero on the chart (under 0.00001s) even at n=40, demonstrating the transformative power of eliminating redundant computation. The gap between recursive and iterative at n=40 is over 3 million times.

6.5 Fibonacci Memory Usage vs n



Observations: Recursive Fibonacci shows near-zero measured memory because tracemalloc only tracks heap allocations, not CPython call stack frames. DP uses slightly more memory (1.63KB at $n=40$) due to the memo dictionary, which is the expected $O(n)$ overhead. Iterative uses effectively zero memory ($O(1)$), confirming it as the most space-efficient approach.

6.6 Visualization Code

All five graphs were produced by `analysis/visualizations.py` using matplotlib. Sorting plots use a log-scale y-axis; Fibonacci plots use a linear scale.

analysis/visualizations.py:

```
"""
Visualization Utilities for Algorithm Performance Analysis

This module provides functions for creating graphical representations
of algorithm performance metrics using matplotlib.
"""

import matplotlib.pyplot as plt
import pandas as pd
import os

def ensure_output_dir():
    """Ensure the graphs output directory exists."""
    os.makedirs("graphs", exist_ok=True)

def plot_sorting_time_comparison(df):
    """
    Plot execution time comparison for sorting algorithms (log scale).

    Args:
        df: DataFrame with columns 'Algorithm', 'Input Size', 'Input Type',
            'Execution Time (s)'
    """
    ensure_output_dir()
    input_types = df["Input Type"].unique()

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    fig.suptitle("Sorting Algorithm Execution Time vs Input Size (Log Scale)", fonts

    for idx, input_type in enumerate(input_types):
```

```

ax = axes[idx]
subset = df[df["Input Type"] == input_type]

for algo in subset["Algorithm"].unique():
    algo_data = subset[subset["Algorithm"] == algo]
    ax.plot(
        algo_data["Input Size"],
        algo_data["Execution Time (s)"],
        marker="o",
        label=algo,
    )

```

```

ax.set_xlabel("Input Size (n)")
ax.set_ylabel("Execution Time (seconds)")
ax.set_title(f"Input: {input_type.capitalize()}")
ax.set_yscale("log")
ax.legend()
ax.grid(True, which="both", linestyle="--", alpha=0.7)

```

```

plt.tight_layout()
plt.savefig("graphs/sorting_time_comparison.png", dpi=150, bbox_inches="tight")
plt.close()
print("Saved: graphs/sorting_time_comparison.png")

```

```

def plot_sorting_memory_comparison(df):

```

```

    """

```

```

    Plot memory usage comparison for sorting algorithms (log scale).

```

```

    Args:

```

```

        df: DataFrame with columns 'Algorithm', 'Input Size', 'Input Type',
            'Memory Usage (KB)'

```

```

    """

```

```

    ensure_output_dir()

```

```

    input_types = df["Input Type"].unique()

```

```

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

```

```

    fig.suptitle("Sorting Algorithm Memory Usage vs Input Size (Log Scale)", fontsize=12)

```

```

    for idx, input_type in enumerate(input_types):

```

```

ax = axes[idx]
subset = df[df["Input Type"] == input_type]

for algo in subset["Algorithm"].unique():
    algo_data = subset[subset["Algorithm"] == algo]
    ax.plot(
        algo_data["Input Size"],
        algo_data["Memory Usage (KB)"],
        marker="s",
        label=algo,
    )

ax.set_xlabel("Input Size (n)")
ax.set_ylabel("Peak Memory (KB)")
ax.set_title(f"Input: {input_type.capitalize()}")
ax.set_yscale("log")
ax.legend()
ax.grid(True, which="both", linestyle="--", alpha=0.7)

plt.tight_layout()
plt.savefig("graphs/sorting_memory_comparison.png", dpi=150, bbox_inches="tight")
plt.close()
print("Saved: graphs/sorting_memory_comparison.png")

def plot_sorting_comparisons(df):
    """
    Plot comparison count for sorting algorithms (log scale).

    Args:
        df: DataFrame with columns 'Algorithm', 'Input Size', 'Input Type',
            'Comparisons'
    """
    ensure_output_dir()
    input_types = df["Input Type"].unique()

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    fig.suptitle("Sorting Algorithm Comparisons vs Input Size (Log Scale)", fontsize

    for idx, input_type in enumerate(input_types):

```



```

ax = axes[idx]
subset = df[df["Input Type"] == input_type]

for algo in subset["Algorithm"].unique():
    algo_data = subset[subset["Algorithm"] == algo]
    ax.plot(
        algo_data["Input Size"],
        algo_data["Comparisons"],
        marker="^",
        label=algo,
    )

```

```

ax.set_xlabel("Input Size (n)")
ax.set_ylabel("Number of Comparisons")
ax.set_title(f"Input: {input_type.capitalize()}")
ax.set_yscale("log")
ax.legend()
ax.grid(True, which="both", linestyle="--", alpha=0.7)

```

```

plt.tight_layout()
plt.savefig("graphs/sorting_comparisons.png", dpi=150, bbox_inches="tight")
plt.close()
print("Saved: graphs/sorting_comparisons.png")

```

```

def plot_fibonacci_time_comparison(df):
    """
    Plot execution time comparison for Fibonacci algorithms.

    Args:
        df: DataFrame with columns 'Algorithm', 'n', 'Execution Time (s)'
    """
    ensure_output_dir()

    plt.figure(figsize=(10, 6))

    for algo in df["Algorithm"].unique():
        algo_data = df[df["Algorithm"] == algo].dropna()
        plt.plot(
            algo_data["n"],

```

```

        algo_data["Execution Time (s)",
        marker="o",
        label=algo,
    )

plt.xlabel("n (Fibonacci Index)")
plt.ylabel("Execution Time (seconds)")
plt.title("Fibonacci Algorithm Execution Time vs n")
plt.legend()
plt.grid(True)
plt.savefig("graphs/fibonacci_time_comparison.png", dpi=150, bbox_inches="tight")
plt.close()
print("Saved: graphs/fibonacci_time_comparison.png")

```

```

def plot_fibonacci_memory_comparison(df):
    """
    Plot memory usage comparison for Fibonacci algorithms.

    Args:
        df: DataFrame with columns 'Algorithm', 'n', 'Memory Usage (KB)'
    """
    ensure_output_dir()

    plt.figure(figsize=(10, 6))

    for algo in df["Algorithm"].unique():
        algo_data = df[df["Algorithm"] == algo].dropna()
        plt.plot(
            algo_data["n"],
            algo_data["Memory Usage (KB)"],
            marker="s",
            label=algo,
        )

    plt.xlabel("n (Fibonacci Index)")
    plt.ylabel("Peak Memory (KB)")
    plt.title("Fibonacci Algorithm Memory Usage vs n")
    plt.legend()
    plt.grid(True)

```

```
plt.savefig("graphs/fibonacci_memory_comparison.png", dpi=150, bbox_inches="tight")
plt.close()
print("Saved: graphs/fibonacci_memory_comparison.png")
```

```
def generate_all_plots(sorting_df, fibonacci_df):
```

```
    """
```

```
    Generate all performance comparison plots.
```

```
    Args:
```

```
        sorting_df: DataFrame with sorting algorithm performance data
```

```
        fibonacci_df: DataFrame with Fibonacci algorithm performance data
```

```
    """
```

```
    print("Generating sorting algorithm plots...")
```

```
    plot_sorting_time_comparison(sorting_df)
```

```
    plot_sorting_memory_comparison(sorting_df)
```

```
    plot_sorting_comparisons(sorting_df)
```

```
    print("Generating Fibonacci algorithm plots...")
```

```
    plot_fibonacci_time_comparison(fibonacci_df)
```

```
    plot_fibonacci_memory_comparison(fibonacci_df)
```

```
    print("\nAll plots saved to graphs/ directory")
```

7. Discussion

7.1 Theoretical vs Experimental Performance

The measured execution times broadly confirm Big-O predictions, but with important nuances.

Where theory matches practice: Bubble Sort and Insertion Sort both exhibit clear $O(n^2)$ growth - at $n=10000$ reverse-sorted, Bubble takes 5.56s and Insertion takes 3.41s, both consistent with quadratic scaling. Merge Sort stays within 0.019-0.024s across all input sizes and types, confirming its guaranteed $O(n \log n)$ behavior. The recursive Fibonacci explodes exponentially (6.70s at $n=40$), while iterative/DP stay under 0.00001s.

A key deviation - Quick Sort's randomized pivot: The pseudocode in Section 3.4 shows last-element pivot selection, which has $O(n^2)$ worst case on sorted input. However, the actual implementation uses randomized pivot selection. This explains why Quick Sort on sorted input (0.014s at $n=10000$) shows no $O(n^2)$ blowup - it performs identically to random input (0.013s). The randomized pivot makes worst-case behavior a statistical improbability rather than a deterministic outcome, which is a practical deviation from the theoretical worst case that the assignment asks us to note.

Constant factors matter: Both Bubble Sort and Insertion Sort are $O(n^2)$, yet Insertion Sort is consistently ~39% faster (3.41s vs 5.56s at $n=10000$ reverse). This is because Insertion Sort performs fewer assignments per comparison (shifts instead of swaps) and has better cache locality from its sequential memory access pattern. Same complexity class, different constant factor - a reminder that Big-O describes growth rate, not absolute speed.

7.2 Scalability

Algorithm	Practical Limit	Reason
Bubble/Insertion	$n \sim 10,000$	$O(n^2)$ becomes minutes at $n=50,000+$
Merge/Quick	$n \sim 1,000,000+$	$O(n \log n)$ stays under seconds
Fibonacci Recursive	$n \sim 40$	Exponential time, 6.7s at $n=40$
Fibonacci Iterative/DP	$n \sim 1,000,000+$	$O(n)$ linear growth

Bubble Sort at $n=10000$ already takes 5.56 seconds; scaling to $n=100000$ would take approximately 556 seconds (9+ minutes). Merge Sort at the same size takes 0.02 seconds - it could handle $n=1000000$ in roughly 0.3 seconds. For Fibonacci, recursive computation becomes impractical beyond $n=40$ (6.7s), while iterative computes $n=1000000$ in under a second.

7.3 Time-Space Trade-offs

Merge Sort guarantees $O(n \log n)$ time but requires $O(n)$ auxiliary space - at $n=10000$ it uses 273KB versus Quick Sort's 78KB. Quick Sort achieves $O(\log n)$ space through in-place partitioning but sacrifices the time guarantee. For Fibonacci, DP achieves $O(n)$ time like iterative but uses $O(n)$ memo space (1.63KB at $n=40$) versus iterative's $O(1)$. In practice, the memory savings of iterative Fibonacci are negligible at small n but become significant at very large n .

7.4 Input Characteristics Impact

Sorted input is the best case for Bubble/Insertion Sort due to early-exit optimization: Bubble Sort on sorted data at $n=10000$ takes only 0.0014s ($O(n)$ comparisons), versus 5.56s on reverse-sorted ($O(n^2)$). Reverse-sorted input is the worst case for both. For $O(n \log n)$ algorithms, input order has minimal impact: Merge Sort varies by only 28% between sorted (0.019s) and random (0.024s) at $n=10000$, and Quick Sort shows even less variation thanks to randomized pivoting.

7.5 Strengths and Weaknesses

Algorithm	Strengths	Weaknesses
Bubble Sort	Simple, stable, adaptive	Slow for large datasets
Insertion Sort	Simple, online, fast on small data	Slow for large datasets
Merge Sort	Guaranteed $O(n \log n)$, stable	$O(n)$ extra space
Quick Sort	Fast in-place, cache-friendly	Unstable, theoretical $O(n^2)$ worst case
Fibonacci Recursive	Simple, mirrors definition	Exponential time, impractical
Fibonacci Iterative	Fast, $O(1)$ space	More code than recursive
Fibonacci DP	Fast, cache for reuse	$O(n)$ space

8. Conclusion

8.1 Quick Sort is the most efficient sorting algorithm in practice (0.013s at $n=10000$ random). Merge Sort is the most consistent, with identical performance regardless of input order.

8.2 Iterative Fibonacci is the most efficient Fibonacci implementation ($O(n)$ time, $O(1)$ space). Naive recursion is impractical beyond $n=40$.

8.3 Key Lessons:

1. Big-O describes growth rate, not absolute speed - constant factors and input characteristics matter
2. Randomized pivot selection transforms Quick Sort from $O(n^2)$ worst case to near-guaranteed $O(n \log n)$
3. Space-time trade-offs are real: Merge Sort's guaranteed time comes at the cost of $O(n)$ memory
4. Profiling reveals what theory predicts but cannot measure - actual seconds, actual kilobytes
5. No single algorithm is best for all scenarios - the right choice depends on data size, order, and constraints

8.4 Recommendations:

- Small datasets ($n < 50$): Insertion Sort - simple, fast, low overhead
- Large datasets: Merge Sort for stability and predictability, Quick Sort for raw speed
- Fibonacci: Always use iterative or DP - never naive recursion for $n > 30$
- Online/streaming data: Insertion Sort - processes elements as they arrive
- Memory-constrained: Quick Sort ($O(\log n)$) or Iterative Fibonacci ($O(1)$)